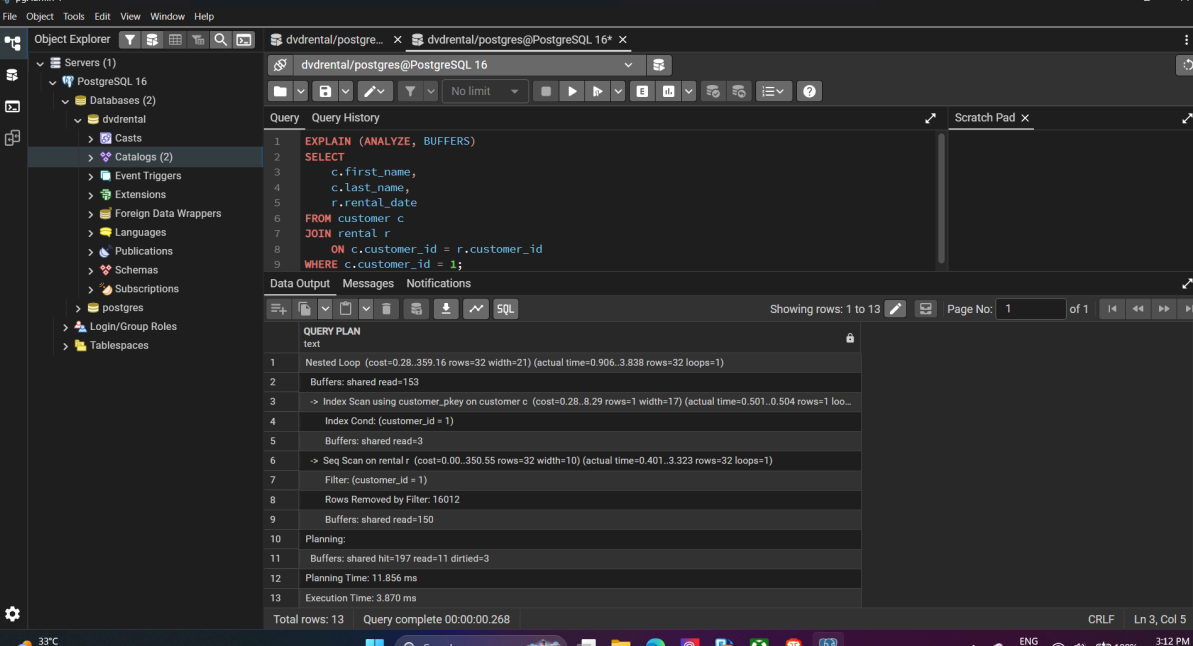


Lab Exercise 1: Triggering Nested Loop Join



The screenshot shows the PostgreSQL Query Editor interface. The query being executed is:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   c.first_name,
4   c.last_name,
5   r.rental_date
6 FROM customer c
7 JOIN rental r
8   ON c.customer_id = r.customer_id
9 WHERE c.customer_id = 1;
```

The Query Plan output is as follows:

Step	Operation	Cost	Actual Time	Actual Rows	Width	Loops
1	Nested Loop	0.28..359.16	0.906..3.838	32	21	1
2	Buffers: shared read	153				
3	Index Scan using customer_pkey on customer c	0.28..8.29	0.501..0.504	1	17	1
4	Index Cond: (customer_id = 1)					
5	Buffers: shared read	3				
6	Seq Scan on rental r	0.00..350.55	0.401..3.323	32	10	1
7	Filter: (customer_id = 1)					
8	Rows Removed by Filter	16012				
9	Buffers: shared read	150				
10	Planning:					
11	Buffers: shared hit	197				
12	Planning Time	11.856 ms				
13	Execution Time	3.870 ms				

Total rows: 13 Query complete 00:00:00.268

- Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?
→Answer: The 'customer' table acts as the Outer (driving) table, while the 'rental' table acts as the Inner table. The predicate 'c.customer_id = 1' reduces the customer relation to only one row

- Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?
→Answer: The 'customer' table returns only one row because 'customer_id' uniquely identifies a customer.

A Nested Loop is therefore efficient because PostgreSQL only performs one lookup against the inner rental table.

A Hash Join would require additional work to build a hash table and scan/probe the inputs. For such a highly selective lookup, that overhead is unnecessary.

- Efficiency: Look at loops in the Inner Table node. Why is it 1?
→Answer: The Inner Table is executed once for every row produced by the Outer Table. Since the Outer 'customer' table produces: 1 row the Inner 'rental' node is executed: 1 time
Therefore: loops = 1